An Open-Source Plug-and-Play Testbed for SDN Controller Performance Benchmarking

Tamerlan Abdullayev
Department of Computer Science
University of Cincinnati
Cincinnati, OH, USA
abdulltn@mail.uc.edu

Giovani Abuaitah
Department of Computer Science
University of Cincinnati
Cincinnati, OH, USA
abuaitgi@ucmail.uc.edu

***Abstract*—Software-Defined Networking (SDN) moves forwarding decisions from individual switches to a central software controller, so the controller's speed directly affects network performance. Measuring that speed reliably is harder than it seems. Existing benchmarking tools test controllers with artificial request streams rather than real network traffic, while studies that emulate real networks are usually run by hand, rarely repeat their experiments, and seldom report how much results vary between runs, so their findings are difficult to reproduce or compare. We present an open source testbed that closes this gap by automating the full benchmarking process. It emulates a network of switches and hosts, sends real TCP traffic between the hosts, and records how long the controller takes to handle each request a switch sends it about a new flow. Every experiment is driven by a random seed, so its traffic can be regenerated exactly; each configuration is repeated to quantify variability; and every run is checked to confirm that it delivered its configured traffic, so that results distorted by an overloaded emulation machine are detected rather than reported. Because the testbed records every individual response time, it shows not only how fast a controller typically responds but also how slow its slowest responses become, which averages and round trip delay measurements hide. New controllers and network layouts can be added through small, documented modules without changing the rest of the testbed.**

***Index Terms*—SDN, benchmarking, testbed, reproducibility, network emulation, OpenFlow, controller performance, open source.**

I. Introduction

Software-defined networking (SDN) [1] decouples the control plane from the data plane, placing forwarding intelligence in a software controller. As SDN deployments scale [2], rigorous and reproducible benchmarking of controller performance becomes increasingly important for SDN research, controller selection, scalability assessment, and capacity planning.

The Reproducibility Gap. Despite the availability of several benchmarking tools, the SDN research community lacks an open-source, end-to-end testbed that is both systematic and reproducible across studies. Synthetic tools such as CBench [3] emulate switches that send Packet-In messages to the controller but generate no end-to-end TCP flows over an emulated topology. RFC 8456 [4] recommends evaluation across multiple topologies and network sizes with repeated trials, but, to the best of our knowledge, open-source testbeds that implement these recommendations in an automated, reusable manner remain limited. Emulation studies using Mininet and iPerf [5]–[7] typically measure a single host pair per scenario and report neither run-to-run variability nor per-event controller processing time, which makes their findings challenging to verify or reuse.

Our Contribution. We present an open-source, plug-and-play SDN benchmarking testbed[1] that addresses these gaps. Its contributions are: (i) a seed-controlled, fully automated pipeline that orchestrates experiment execution, network emulation, traffic generation, result aggregation, and cleanup, in which the traffic schedule of every run can be regenerated from its seed, repetitions with distinct seeds quantify run-to-run variability, and full sweeps run without manual intervention; (ii) per-event controller instrumentation that records the processing time of every Packet-In handler invocation with a high-resolution monotonic clock, enabling separate characterization of median (p50) and tail (p95) controller-side latency, a distinction not possible with the RTT-based measurements used in prior work; (iii) end-to-end TCP traffic with verified delivery, in which rate-paced TCP flows are launched at a configurable rate between randomly selected host pairs, so that Packet-In events originate from real host-to-host connections across multiple switches rather than from synthetic stimulus, and every run reports its flow success rate and achieved flow rate, so that results obtained under failed or delayed traffic generation can be detected rather than silently aggregated; and (iv) plug-and-play extensibility, with flexible configuration of network topologies and traffic patterns, new topologies added as modules, and new controllers integrated through a documented adapter contract, without modifying the rest of the pipeline.

II. Background and Related Work

A. SDN and OpenFlow

In SDN, the forwarding devices, called *switches* in OpenFlow whether they forward at layer 2 or layer 3, hold flow tables of rules that map packets to forwarding actions, and a software controller decides which rules to install. The OpenFlow protocol [1], [8] provides a standard communication interface between the controller and the switches. When a

[1]The testbed is available at https://github.com/tamerlanabdullayev/sdn-testbed.

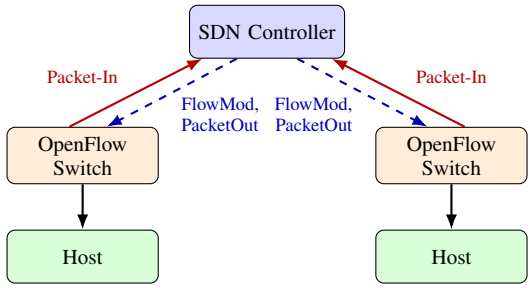


Fig. 1. SDN control interactions over OpenFlow: a switch sends a Packet-In to the controller for a packet that matches no rule, and the controller replies with a FlowMod and a PacketOut.

packet matches no rule in a switch's flow table, the switch sends a *Packet-In* message to the controller, which replies with a *FlowMod* that installs a rule for the flow and a *PacketOut* that tells the switch how to forward the packet (Fig. 1). Later packets of the same flow are forwarded by the switch without involving the controller, so the time the controller needs to handle each Packet-In determines how quickly new flows are set up.

B. Synthetic Benchmarking Tools

The most widely used tool for SDN controller benchmarking is CBench [3], which emulates a set of OpenFlow switches that send Packet-In messages to the controller and measures the rate of returned FlowMod responses. While widely cited, CBench emulates switches without modeling inter-switch connectivity, supports only OpenFlow 1.0 [9], and does not generate end-to-end TCP flows between hosts; it therefore may not fully capture the topology-driven event dynamics that arise in realistic deployments. Zhu et al. [9] qualitatively classified 34 controllers and benchmarked nine of them using CBench, PktBlaster, and OFNet, noting that *none of the tools available can measure all performance statistics* and that performance metrics are often simplified or interpreted inconsistently across tools and prior studies. OFLOPS [10] targets switch rather than controller evaluation, while WCBench [11] (tied to a specific OpenDaylight release) and HCprobe [12] extend CBench but retain its synthetic traffic model.

C. IETF Benchmarking Standards and Gaps

The IETF Benchmarking Methodology Working Group (BMWG) has published several relevant informational RFCs. RFC 8456 [4] defines a methodology for benchmarking SDN controller performance, recommending that controllers be evaluated across multiple network topologies and a varying number of network devices, and that each test be repeated a minimum of 10 times to increase confidence in the results, with averages and variances reported across trials. RFC 5180 [13] notes that the device under test can be monitored for processor and memory utilization. Our testbed draws on both, with multi-topology sweeps of 10 repetitions per configuration and per-second controller CPU and memory monitoring. These documents, however, address methodology only and do not prescribe open-source tooling or reference implementations.

TABLE I
COMPARISON OF SDN BENCHMARKING APPROACHES

Feature	CBench [3]	Mininet studies†	Our Testbed
Emulated inter-switch topology	✗	✓	✓
End-to-end TCP traffic	✗	✓	✓
Traffic delivery validation	✗	✗	✓
Automated end-to-end pipeline	○	✗	✓
Factorial parameter sweep	○	○	✓
Repeated trials w/ variability	✓	○	✓
Per-event controller latency	✗	✗	✓
CPU/memory monitoring	✗	✗	✓
Public source code	✓	✗	✓
Pluggable topology module	✗	✗	✓

✓ = supported; ○ = partial; ✗ = absent or not reported. † [5]–[7].

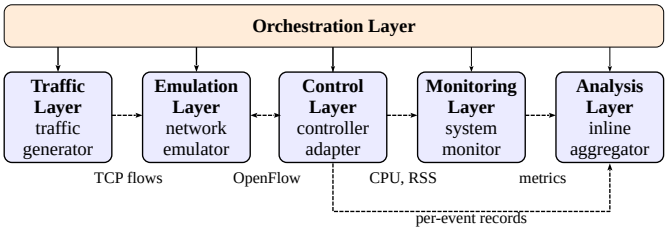


Fig. 2. Testbed architecture. The orchestration layer coordinates all components; the aggregator consolidates per-run outputs.

D. Academic Studies Using Mininet and iPerf

Several recent studies use Mininet [14] and iPerf but observe controller behavior primarily through end-to-end metrics. Koulouras et al. [5] compared four controllers on k-ary fat-trees (k=4–10), measuring throughput, RTT, and jitter between one host pair per scenario, without reporting repeated trials. Čabarkapa et al. [6] evaluated Ryu and ONOS on a single fat-tree (32 hosts) between one host pair, inspecting OpenFlow message rates with Wireshark, but did not vary network size or load. Ali [7] compared four controllers across five topologies using 50 iPerf and ping packets per configuration, without reporting variability. None of these studies report a public code release. Table I summarizes the methodological gaps addressed by our testbed.

III. TESTBED DESIGN

A. Overview

Fig. 2 illustrates the testbed architecture. Four components interact under the control of an automated orchestration layer: (1) the *instrumented controller adapter*, (2) the *network emulator*, (3) the *traffic generator*, and (4) the *system monitor*. Each component is defined by the interface it exposes rather than by a specific tool, so any of them can be replaced without modifying the others.

B. Controller Adapter Contract

A controller is integrated through an *adapter*, which must provide three things. First, *lifecycle control*: the adapter starts the controller on a given OpenFlow port with a given per-run output directory, reports its process identifier for monitoring,

and stops it cleanly. Second, a *reference control application* that defines the workload: MAC-learning L2 switching in which flow entries match on the input port and the source and destination MAC addresses. Flow entries are installed without idle or hard timeouts, so each directed host pair triggers Packet-In events at a given switch only until its entry is installed, after which flows between that pair are forwarded without controller involvement. Third, *instrumentation* that emits two records in fixed schemas:

- a *per-event record* for every Packet-In, containing the switch identifier, source and destination addresses, and the *handler processing time*: the elapsed time from the moment the controller application begins handling the Packet-In until it has issued its response—a FlowMod followed by a PacketOut for a known destination, or a flooding PacketOut otherwise; and
- a *per-second record* of Packet-In and FlowMod counts, produced without blocking the controller's event loop.

The handler processing time isolates controller-side processing from network and switch delays, but excludes the time a message spends in transit and waiting in the controller's event queue before dispatch; queue residence time is a complementary metric that can be obtained without controller changes by timestamping OpenFlow messages on the control channel, which we leave to future work. Unlike the black-box measurement model of RFC 8456 [4], which observes timing at the test emulator interfaces, these timestamps are taken inside the controller application. Reporting both the median (p50) and 95th-percentile (p95) of these per-event times is important because the mean can mask tail behavior in event-driven systems under bursty load, as queueing analyses of SDN controllers [15] suggest.

Integrating a new controller therefore amounts to implementing the lifecycle operations and emitting these two records; the emulator, traffic generator, monitor, orchestration, and aggregation remain unchanged.

C. Topology Module

Topologies are modules that expose a single builder parameterized by the host count N and are registered by name, so a new topology can be added without modifying the rest of the pipeline. Static address resolution is configured on all hosts, so no ARP traffic reaches the controller. The testbed currently includes two loop-free topologies used in this paper: a star (N hosts on a single switch) and a two-tier tree, in which one core switch connects $\lceil N/4 \rceil$ edge switches with up to four hosts each. An experimental full-mesh topology is also included but requires a loop-prevention mechanism (e.g., STP) in the control application and is not evaluated here.

D. Traffic Model

Flows are launched at a configured rate (FPS) between pairs of distinct hosts chosen uniformly at random from a seeded generator, so the traffic schedule of a run depends only on its seed. Each flow is a TCP transfer of fixed duration d, paced to a fixed per-flow rate. Fixing the per-flow rate makes the offered load depend on the number of flows in flight (FPS $\times$ d) rather than on the host count, and keeps traffic generation from contending with the controller for CPU on a single emulation host. Every destination provides enough receivers that concurrent flows are not rejected. For each run, the traffic generator records the achieved launch rate, and every flow's outcome is classified, so that runs whose workload was not delivered as configured can be detected rather than silently aggregated.

E. System Monitor and Orchestration

For each run, the orchestration layer cleans residual emulator state, starts the controller through its adapter, starts the system monitor—a separate process that samples the controller's CPU utilization, resident memory (RSS), and thread count once per second—launches the emulated network and the traffic schedule, waits for all flows to complete, stops all components, aggregates the run, and prunes bulky raw outputs once aggregation has succeeded. The seed of each run is derived deterministically from its parameters, so any individual run can be regenerated from its parameter tuple, and full factorial sweeps over all parameter combinations run unattended.

F. Metrics and Aggregation

After each run, the inline aggregator computes and appends to the results table: the *flow success rate* (fraction of planned flows completed without error) and *achieved flow rate* (flows launched divided by actual launch duration), which validate that the run delivered its configured workload; *delivered throughput* (bytes sent by successful flows divided by run duration); the mean *Packet-In rate*; the p50 and p95 *handler processing time* over all Packet-In events, including flooding events; and mean controller *CPU* and *memory*. Per-second means include controller startup.

IV. Implementation

A. Network Emulation and Topologies

The network emulator is Mininet [14] with Open vSwitch [16] and TCLink links; static ARP entries are pre-populated with Mininet's `autoStaticArp` option. Topologies are Python classes inheriting from Mininet's `Topo` base class: adding one requires implementing a single `build(self, n)` method and a one-line entry in the topology factory.

B. Traffic Generation

Traffic is generated with iperf3 [17]. Because an iperf3 server serves one test at a time, each host runs a pool of servers on a dedicated block of ports. The pool size is derived from the expected number of concurrent flows per destination, $\lambda = \text{FPS} \cdot (d+g)/N$ with a guard interval g=1 s, as $\lceil 1.5\lambda + 3\sqrt{\lambda} \rceil$ (bounded to 4–320), and each flow is assigned the least-recently-used port on its destination; port assignment is deterministic, so it does not alter the seeded schedule. Traffic begins only after every server port is verified to be listening. Launches follow a wall-clock schedule with sleep

compensation, and finished clients are reaped. Pacing uses kernel socket pacing (`--fq-rate`), avoiding the CPU cost of application-level rate limiting; each client's JSON output is classified as successful, rejected, or failed.

C. Monitoring, Orchestration, and Aggregation

The system monitor uses `psutil` [18]; CPU percentage is primed with an initial non-blocking call before sampling begins, and the monitor runs as a separate OS process to avoid GIL contention with the controller. Orchestration is script-based: the shell script `run_experiment.sh` accepts the topology, host count, flow rate, run duration, flow duration, and seed as command-line arguments and implements the per-run sequence of Section III. Seeds are computed as seed $= B_T + 10N + \text{FPS} + \text{rep}$, where B_T is a per-topology offset (1000 for star, 2000 for the two-tier tree). Matrix scripts iterate over all (topology, N, FPS, repetition) combinations, and the aggregator (`aggregate_results.py`) appends one row per run to a summary CSV.

V. Testbed Validation: Case Studies with Ryu and OS-Ken

A. Controller Adapters

The reference adapter is a Ryu [19] application (v4.34, OpenFlow 1.3) implementing the control logic and instrumentation of Section III. Each Packet-In handler invocation is timestamped on entry with `time.perf_counter()`, a high-resolution monotonic clock, and the handler processing time is written in milliseconds to `flow_events.csv`. Per-second Packet-In and FlowMod counts are written to `controller_metrics.csv` by a background greenthread (`hub.spawn`), so the logger does not block the main event loop. The per-run output directory is passed through the `OUTDIR` environment variable, so the orchestrator can isolate each run's output without modifying controller code. A second adapter for OS-Ken is described in Section V-F.

B. Experimental Configuration

We run a full factorial experiment with the Ryu adapter over two topologies (star, two-tier tree), $N \in \{4, 8, 16, 32, 64\}$ hosts, and flow rates of 25, 50, 100, and 200 flows/s, with 2 s flows paced at 2 Mbit/s over 20 s runs and 10 repetitions with distinct seeds per configuration (400 runs per controller). All experiments run on a single VirtualBox VM (Ubuntu 24.04, 16 vCPUs, 16 GB RAM), executed unattended by the matrix scripts. Each run, including emulator setup and teardown, takes 30–45 s of wall-clock time, so a 400-run sweep completes without intervention in about 4–5 h.

C. Workload Validation

Before controller metrics are analyzed, the testbed verifies that every run delivered its configured workload. Across all 800 runs, the flow success rate was at least 99.8% and the achieved flow rate was within 1.8% of the configured rate. Measured throughput scales linearly with flow rate (131 Mbps at 25 fps to 1.04 Gbps at 200 fps) and differs by less than 0.3% between topologies, confirming that both topologies receive the same offered load. Because the measure uses iperf3's sender-side byte count, which includes data still queued in the socket buffer when a flow ends, its absolute value is an upper bound, about 30% above the nominal paced load. These checks matter: in preliminary runs with unpaced traffic, emulation-host saturation inflated the star's p95 handler processing time at N=64 to ∼18 ms, about 27× the 0.67 ms measured with paced traffic—a distortion that controller-side metrics alone cannot distinguish from genuine controller behavior. CPU averages discard each run's first monitor sample, which covers only a sub-millisecond window after the priming call and occasionally read several thousand percent.

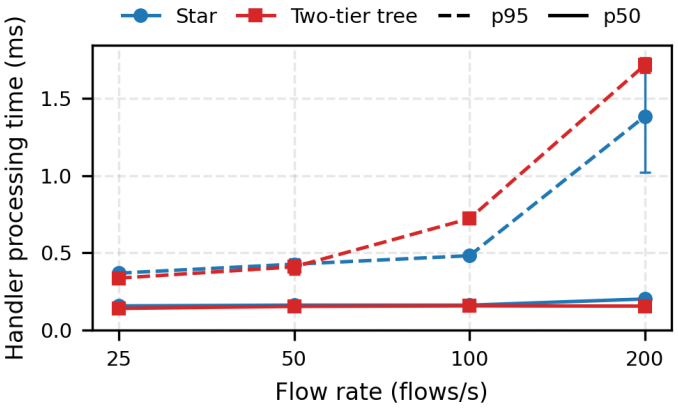


Fig. 3. Ryu, p50 (solid) and p95 (dashed) handler processing time vs. flow rate at N=64. Error bars show 95% confidence intervals of the mean over 10 repetitions.

D. Per-Event Processing Latency

Fig. 3 shows per-event handler processing time at p50 and p95 across flow rates at N=64. The p50 stays nearly constant at 0.14–0.20 ms for both topologies, so a median-based measurement would suggest that load has little effect. The p95, however, rises from 0.37 to 1.38 ms for the star and from 0.34 to 1.71 ms for the tree between 25 and 200 fps, with the steepest increase above 100 fps. To our knowledge, existing open-source benchmarking tools do not report this per-event distribution, and the Mininet-based studies of Section II rely on end-to-end RTT, which conflates network latency with controller processing.

The sweep also exposes how workload parameters interact. Because flow entries are installed without timeouts and ARP is pre-populated, Packet-Ins occur only on first contact between each directed host pair at each switch. At small N, all pairs are discovered early in a run and flow rate has little effect (star, N=4: p95 of 0.39 ms at both 25 and 200 fps); at N=64, many of the 4,032 directed pairs remain unseen even at 200 fps, so higher flow rates keep raising controller load. The N=4 case also shows that traffic volume alone does not inflate the tail, since as many flows are in flight as at N=64.

E. Scalability with Host Count

Fig. 4 and Table II show that p95 grows by 1.7× from N=4 to N=64 for both topologies (star: 0.39 to 0.67 ms; tree:

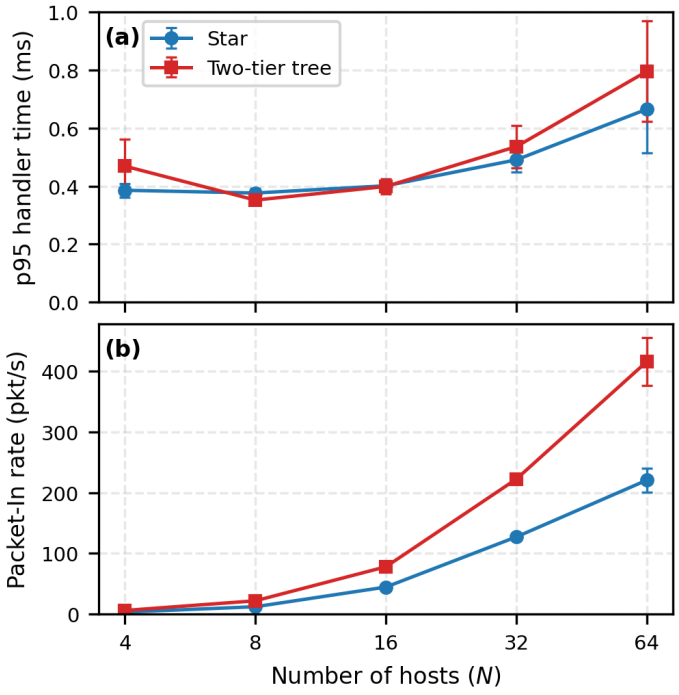


Fig. 4. Ryu, (a) p95 handler processing time and (b) Packet-In rate vs. host count, averaged over flow rates. Error bars show 95% confidence intervals of the mean. Tail latency grows with host count for both topologies, tracking Packet-In load.

0.47 to 0.80 ms), with most of the growth above $N{=}16$; the tree's higher p95 at $N{=}4$ reflects the few Packet-In events per run at this scale. Over the same range, delivered throughput stays at about 490 Mbps (averaged over flow rates), while mean controller CPU rises from about 1% to 10% (star) and 16% (tree), and never exceeds 22% in any configuration, suggesting that the tail is not compute-bound.

The tree generates 1.7–2.1× more Packet-Ins than the star at the same flow rate, because a flow between hosts on different edge switches traverses three switches, each raising its own Packet-In. Although the tree spreads these over $\lceil N/4 \rceil + 1$ switches, its p95 is 1.2× higher at $N{=}64$ (confidence intervals overlap), so per-switch concentration does not explain the tail. Nor does the mean Packet-In rate alone: for the star at $N{=}64$, p95 nearly triples from 100 to 200 fps while the mean Packet-In rate does not increase (283 vs. 269 pkt/s), pointing to the burstiness of Packet-In arrivals, which the testbed's per-second counters can quantify.

F. Extensibility and Reproducibility

Extending the testbed requires little code: the star topology module is 8 lines of Python and the two-tier tree under 20, and the Ryu adapter, including its instrumentation, is about 120 lines. To test adapter portability, we substituted OS-Ken—the actively maintained community fork of Ryu [19], sharing the same event-dispatch architecture and API—and executed the identical 400-run sweep. The adapter differs only in seven `import` lines (`ryu.*` to `os_ken.*`) and its base class (`RyuApp` to `OSKenApp`); apart from the controller launch command, process names, and output paths in the run script, all pipeline components—traffic generator, system monitor, and aggregator—remained unmodified.

TABLE II
RYU METRICS BY TOPOLOGY AND HOST COUNT (MEANS OVER FLOW RATES AND REPETITIONS)

Topo.	N	Tput (Mbps)	Pkt-In (pkt/s)	p95 Lat (ms)	CPU (%)
Star	4	491	3	0.39	0.9
Star	8	491	12	0.38	1.3
Star	16	491	45	0.40	2.6
Star	32	491	127	0.49	6.2
Star	64	490	221	0.67	10.4
Tree	4	489	6	0.47	1.0
Tree	8	491	22	0.35	1.6
Tree	16	491	78	0.40	3.6
Tree	32	491	222	0.54	8.9
Tree	64	488	416	0.80	15.7

TABLE III
RYU VS. OS-KEN (MEANS OVER ALL RUNS; P95 WITH 95% CONFIDENCE INTERVAL)

Ctrl.	Topo.	p95 Lat (ms)	Pkt-In (pkt/s)	CPU (%)	RSS (MB)
Ryu	Star	0.46 ± 0.04	81.6	4.3	58.9
Ryu	Tree	0.51 ± 0.05	148.8	6.2	59.1
OS-Ken	Star	0.41 ± 0.02	83.5	4.0	58.2
OS-Ken	Tree	0.45 ± 0.05	149.9	5.7	58.3

Table III summarizes the results for both controllers. Packet-In rates agree within 3% on average, confirming that both adapters receive the same workload, and OS-Ken reproduces the load-dependent tail: at $N{=}64$, its p95 rises from 0.36 to 0.97 ms (star) and from 0.30 to 1.83 ms (tree) between 25 and 200 fps. OS-Ken's handler processing time is consistently lower, by 12–24% at p50 in every configuration and by 11% at p95 on average (significant in 27 of 40 configurations; Welch's t-test, $p < 0.05$). Both applications share the same control logic, but OS-Ken (v2.8.1) ran under Python 3.12 and Ryu under Python 3.10, so the difference likely reflects the interpreter and dependency versions rather than the framework; we did not isolate it. Mean CPU and RSS differ by less than run-to-run variability and about 1%, respectively. The adapter contract is thus sufficient to onboard a closely related controller without altering any other pipeline component, and the testbed resolves median differences of a few tens of microseconds between controllers.

Repeated runs are consistent. Across the 40 Ryu configurations, the 95% confidence interval of the mean is within 10% of the mean for p50 in all configurations, for the Packet-In rate in 39, and for p95 in 30; the widest intervals occur at $N{=}4$, where few Packet-Ins are sampled per run, and for the star at $N{=}64$ and 200 fps.

VI. DISCUSSION

A. Lessons for Controller Benchmarking

Three lessons from the validation apply beyond the evaluated controllers. First, controller metrics are trustworthy only if the delivered workload is verified: host saturation produced

a 27× tail inflation that resembled controller behavior (Section V-C). Second, the median hides load-dependent behavior that the per-event p95 exposes and that RTT-based measurement cannot separate from network delay. Third, controller-side counters allow explanations to be tested rather than assumed: the data rule out per-switch concentration as the cause of the tail and point to bursty Packet-In arrival. Because handler processing time is wall-clock time on a shared host, preemption by co-located emulation processes cannot be fully ruled out; pinning the controller to dedicated cores to separate these effects is left to future work.

B. Limitations of the Demonstration

The demonstration evaluates two closely related Python controllers with the same single-threaded event-dispatch model, so it validates the adapter contract only for closely related frameworks; adapters for architecturally different controllers such as ONOS [20] or OpenDaylight are future work.

Emulation in a VM adds timing noise, so absolute values reflect the emulated environment; relative comparisons are more robust. Flow entries are installed without timeouts, so Packet-In load is bounded by unique directed host pairs rather than the offered flow rate; flow-expiry workloads are future work. Flows are paced to a fixed 2 Mbit/s so that traffic generation does not contend with the controller for CPU on the single emulation host (Section V-C), which limits the offered data-plane load. Finally, the evaluated tree lacks the redundant paths of a k-ary fat-tree; multipath topologies require loop prevention in the control application.

VII. Conclusion

We presented an open-source, plug-and-play SDN controller benchmarking testbed that addresses three core limitations of the current landscape: the reliance on synthetic stimulus in established tools such as CBench and its derivatives, the limited automation and variability reporting in academic emulation studies, and the limited availability of open-source tooling that operationalizes key RFC 8456 recommendations in an automated, reusable workflow.

The testbed contributes a seed-controlled automated pipeline, per-event controller instrumentation that separates p50 from p95 Packet-In handler processing time, rate-paced end-to-end TCP traffic with per-run validation of delivered flows and achieved flow rate, and a plug-and-play topology and controller adapter system. An 800-run validation study (400 per controller) using Ryu and OS-Ken across star and two-tier tree topologies confirms that the testbed captures load-dependent controller behavior with quantified run-to-run variability—including a rise in tail latency at scale that median and aggregate RTT-only measurements do not reveal—and that the adapter contract enables substitution of a closely related controller without pipeline modification.

The testbed is publicly available as open-source software; planned extensions include per-host containers, link-level parameters as first-class configuration, flow-expiry workloads, and a web-based interface.

References

[1] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner, "OpenFlow: Enabling innovation in campus networks," *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, pp. 69–74, 2008.

[2] D. Kreutz, F. M. V. Ramos, P. E. Veríssimo, C. E. Rothenberg, S. Azodolmolky, and S. Uhlig, "Software-defined networking: A comprehensive survey," *Proceedings of the IEEE*, vol. 103, no. 1, pp. 14–76, 2015.

[3] Stanford University, "CBench: Controller benchmarking tool," https://github.com/mininet/oflops, 2012.

[4] V. Bhuvaneswaran, A. Basil, M. Tassinari, V. Manral, and S. Banks, "Benchmarking methodology for software-defined networking (SDN) controller performance," Internet Engineering Task Force, RFC 8456, 2018.

[5] I. Koulouras, S. V. Margariti, I. Bobotsaris, E. Stergiou, and C. Stylios, "On the performance of SDN controllers in real world topologies," in *2022 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN)*, 2022, pp. 143–148.

[6] D. Čabarkapa, D. Rančić, P. Pavlović, and M. Milićević, "Investigating the impact of tree-based network topology on the SDN controller performance," *Facta Universitatis, Series: Automatic Control and Robotics*, vol. 21, no. 1, pp. 25–35, 2022.

[7] U. Ali, "Performance comparison of SDN controllers in different network topologies," *Research Square (preprint)*, 2024, dOI: 10.21203/rs.3.rs-4826985/v1.

[8] Open Networking Foundation, "OpenFlow switch specification version 1.3.0," https://www.opennetworking.org, 2012.

[9] L. Zhu, M. M. Karim, K. Sharif, X. Du, and M. Guizani, "SDN controllers: Benchmarking & performance evaluation," *arXiv preprint arXiv:1902.04491*, 2019.

[10] C. Rotsos, N. Sarrar, S. Uhlig, R. Sherwood, and A. W. Moore, "OFLOPS: An open framework for OpenFlow switch evaluation," in *Proceedings of the 13th International Conference on Passive and Active Measurement (PAM)*, 2012.

[11] D. Farrell, "WCBench: CBench, wrapped in stuff that makes it useful," https://github.com/dfarrell07/wcbench, 2015.

[12] A. Shalimov, D. Zuikov, D. Zimarina, V. Pashkov, and R. Smeliansky, "Advanced study of SDN/OpenFlow controllers," in *Proceedings of the 9th Central & Eastern European Software Engineering Conference (CEE-SECR)*, 2013.

[13] C. Popoviciu, A. Hamza, G. V. de Velde, and D. Dugatkin, "IPv6 benchmarking methodology for network interconnect devices," Internet Engineering Task Force, RFC 5180, 2008, iETF Benchmarking Methodology Working Group.

[14] B. Lantz, B. Heller, and N. McKeown, "A network in a laptop: Rapid prototyping for software-defined networks," in *Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks (HotNets-IX)*, 2010.

[15] B. Xiong, K. Yang, J. Zhao, W. Li, and K. Li, "Performance evaluation of OpenFlow-based software-defined networks based on queueing model," *Computer Networks*, vol. 102, pp. 172–185, 2016.

[16] Open vSwitch, "OVS: Production quality, multilayer open virtual switch," https://www.openvswitch.org/, 2026, accessed: 2026-06-08.

[17] J. Dugan, S. Elliott, B. A. Mah, J. Poskanzer, and K. Prabhu, "iperf3: A TCP, UDP, and SCTP network bandwidth measurement tool," https://software.es.net/iperf/, 2014.

[18] G. Rodola, "psutil: Cross-platform lib for process and system monitoring in Python," https://github.com/giampaolo/psutil, 2026, accessed: 2026-06-12.

[19] NTT Software Innovation Center, "Ryu SDN framework," https://github.com/osrg/ryu, 2012.

[20] P. Berde, M. Gerola, J. Hart, Y. Higuchi, M. Kobayashi, T. Koide, B. Lantz, B. O'Connor, P. Radoslavov, W. Snow, and G. Parulkar, "ONOS: Towards an open, distributed SDN OS," in *Proceedings of the Third Workshop on Hot Topics in Software Defined Networking (HotSDN)*, 2014.